

Instructions:

- Make sure your code is clean, well-organized, uses meaningful variable names, includes useful comments, and meets specific efficiency requirements (if any).
- Make sure you edit the sections (#1 mandatory, #2 if applicable, and #3 optional) in the given `notes.txt` file as appropriate. In section #1, for each problem, describe your approach along with any issues you encountered and if/how you managed to solve those issues.

Goal: Become familiar with the Java Virtual Machine (JVM) and extend the *j--* language by adding support for some arithmetic operators, conditional expression, and do statement.

Zip File: Download and unzip the zip file [📄](#) for the assignment under `$j/j--`.

Language Specification: Consult the *j--* Language Specification [📄](#) for the syntactic and semantic rules that you must follow when you make changes to the *j--* language described in the problems below. Pay close attention to any problem-specific exceptions to the language rules, which will be stated in the problem description.

Problem 1. (*Understanding the JVM*) Complete the implementation of the program `GenPrimeCounter.java` that uses the `CLEmitter` interface to create a JVM program called `PrimeCounter.class` that receives `n` (int) as command-line input and writes the number of primes $\leq n$ as output. Use the *j--* program `tests/PrimeCounter.java` to help you with the logic for the boolean `isPrime(int x)` method that checks if `x` is prime or not and the `int primes(int n)` method that returns the number of primes $\leq n$.

```
$ ant
$ ./bin/clemitter simpleops/GenPrimeCounter.java
$ java PrimeCounter 100
25
```

Problem 2. (*Arithmetic Operators*) Implement the division `/`, remainder `%`, and unary plus `+` operators in *j--*.

AST representations to use:

- `JDivideOp` in `JBinaryExpression.java` for `/`.
- `JRemainderOp` in `JBinaryExpression.java` for `%`.
- `JUnaryPlusOp` in `JUnaryExpression.java` for unary `+`.

```
$ ant
$ ./bin/j-- simpleops/Division.java
$ java Division 60 13
4
$ ./bin/j-- simpleops/Remainder.java
$ java Remainder 60 13
8
$ ./bin/j-- simpleops/UnaryPlus.java
$ java UnaryPlus 60
60
```

Problem 3. (*Conditional Expression*) Add support for conditional expression (`e ? e1 : e2`) in *j--*. Here are the grammar rules related to conditional expression:

```
assignmentExpression ::= conditionalExpression
                        [ ( ASSIGN | PLUS_ASSIGN ) assignmentExpression ]

conditionalExpression ::= conditionalAndExpression
                        [ QUESTION expression COLON conditionalExpression ]
```

Use JConditionalExpression.java as the AST representation for a conditional expression.

```
$ ant
$ ./bin/j-- simpleops/ConditionalExpression.java
$ java ConditionalExpression
Tails
$ java ConditionalExpression
Tails
$ java ConditionalExpression
Heads
```

Problem 4. (*Do Statement*) Add support for a do statement in *j--*. Use JDoStatement.java as the AST representation for a do statement.

```
$ ant
$ ./bin/j-- simpleops/DoStatement.java
$ java DoStatement 100
5050
```

Files to Submit:

1. GenPrimeCounter.java
2. JBinaryExpression.java
3. JConditionalExpression.java
4. JDoStatement.java
5. JUnaryExpression.java
6. Parser.java
7. Scanner.java
8. TokenInfo.java
9. notes.txt